LAPORAN PENGEMBANGAN WEB: *ETS UJI BEBAN TOKO KILAT*



Disusun Oleh:

Muhammad Arrafi Reva Razqana Arassy

NIM

241524017

PROGRAM STUDI D4-TEKNIK INFORMATIKA
JURUSAN TEKNIK KOMPUTER DAN INFORMATIKA
POLITEKNIK NEGERI BANDUNG
2026

DAFTAR ISI

DAFTAR ISI 2
I HASIL PEKERJAAN 3
1.1 Tujuan Praktikum 3
1.2 Konfigurasi Lingkungan Pengujian 3
1.3 Tabel Ringkasan Median Pengujian Utama 3
1.3.1 Pengujian Utama 3
1.3.2 Pengujian Kondisi Jaringan 3
1.4 Grafik Hasil Pengujian Utama 4
1.4.1 Grafik 1: Concurrency vs Median p95 Response Time 4
1.4.2 Grafik 2: Concurrency vs Median Throughput (RPS) 4
1.4.3 Grafik 3: Concurrency vs Median Failed Requests 5
1.4.4 Grafik 4a: Concurrency vs Penggunaan CPU (% 1 Core) 5
1.4.5 Grafik 4b: Concurrency vs Penggunaan Memori RSS (MB) 6
1.4.6 Grafik Perbandingan Kondisi Jaringan (N1 vs N2) 6
1.5 Pembahasan dan Jawaban Pertanyaan Evaluasi ETS 7
1.5.1 Q1: Perubahan Average, p50, p95, dan p99 Response Time 7
1.5.2 Q2: Titik Peningkatan p95 Terjelas dan Perubahan RPS 7
1.5.3 Q3: Perubahan Failed Requests 7
1.5.4 Q4: Perubahan CPU Server dan Analisis Bottleneck 7
1.5.5 Q5: Analisis Penggunaan RAM RSS vs Heap Used 8
1.5.6 Q6: Arti Cache Hits, Misses, Hit Ratio dan Efektivitas Performa 8
1.5.7 Q7: Metrik Filesystem Read/Write vs Disk Throughput (MB/s) 8
1.5.8 Q8: Perbandingan Kondisi Jaringan (N1 vs N2) 8
1.5.9 Q9: Analisis Kenaikan Response Time Tanpa Kenaikan CPU 8
1.5.10 Q10: Skenario Paling Sesuai dan Analisis Trade-off 9
1.5.11 Q11: Keterbatasan Pengujian Laboratorium 9
1.6 Kesimpulan 10

I

HASIL PEKERJAAN

1.1 Tujuan Praktikum

Menganalisis skalabilitas, throughput (Requests Per Second / RPS), waktu respons (latency), efektivitas in-memory caching, serta konsumsi sumber daya komputasi (CPU dan RAM) pada layanan API TokoKilat saat menerima peningkatan beban concurrency bertahap (1, 10, 20, dan 50) menggunakan metode eksperimen OFAT (One Factor at a Time).

1.2 Konfigurasi Lingkungan Pengujian

- Perangkat Keras: Laptop Pribadi (macOS, arsitektur Apple Silicon)
- Eksperimen Jaringan & Docker (N1 & N2): Dijalankan pada GitHub Codespaces (Cloud Virtual Machine berbasis Linux Ubuntu, Docker Engine, iproute2/tc, Node.js v20+).
- Alasan Penggunaan GitHub Codespaces: macOS lokal tidak memiliki utilitas kernel Linux tc (traffic control). Penggunaan GitHub Codespaces memungkinkan eksekusi container Docker dengan --network host dan pembatasan jaringan tc netem dilakukan di lingkungan Linux yang sesuai standar praktikum tanpa modifikasi sistem lokal.

1.3 Tabel Ringkasan Median Pengujian Utama

1.3.1 Pengujian Utama

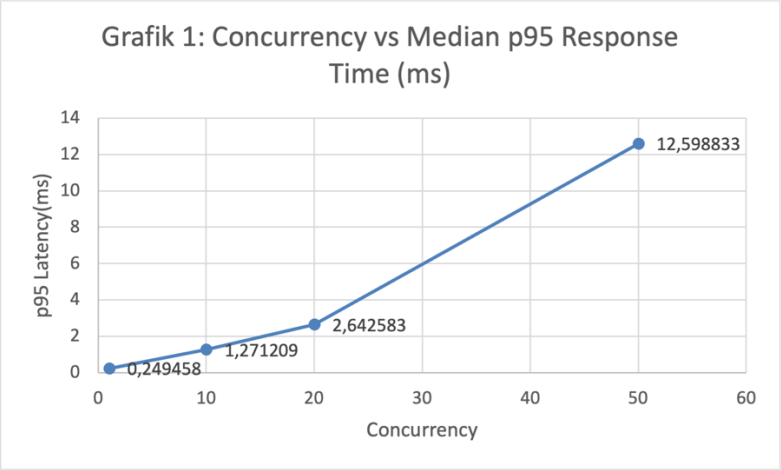
Concurrency	requests_per_second	p95_latency_ms	failed_requests	server_cpu_percent_one_core	server_memory_rss_mb
1	6.342,49	0,249458	0	64,18282424	63,96875
10	12.287,63	1,271209	0	125,5689342	63,375
20	13.071,94	2,642583	0	128,8984605	63,75
50	13.352,66	12,598833	0	130,2605165	64,484375

1.3.2 Pengujian Kondisi Jaringan

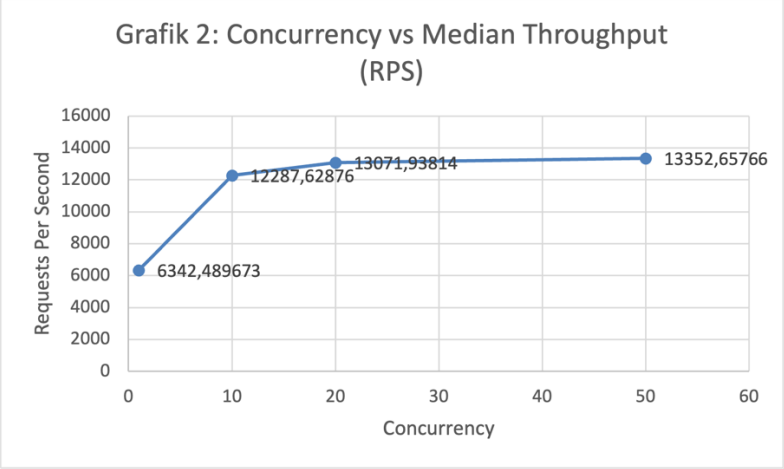
Scenario	Concurrency	requests_per_second	p50_latency_ms	p95_latency_ms	p99_latency_m	failed_requests
N1	20	2334,593098446	7,669034	15,687614	22,859375	0
N2	20	191,20408238383	101,842413	106,020612	125,418557	0

1.4 Grafik Hasil Pengujian Utama

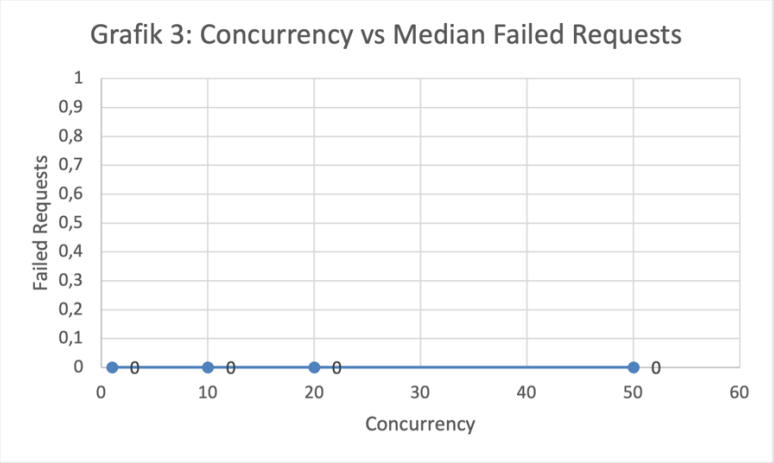
1.4.1 Grafik 1: Concurrency vs Median p95 Response Time



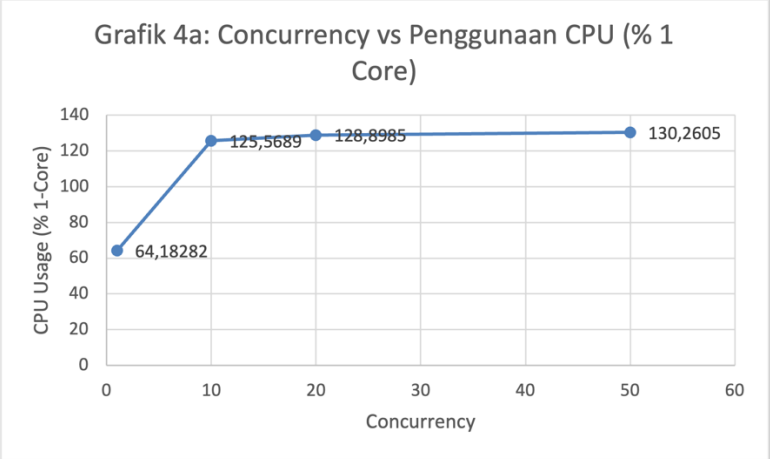
1.4.2 Grafik 2: Concurrency vs Median Throughput (RPS)



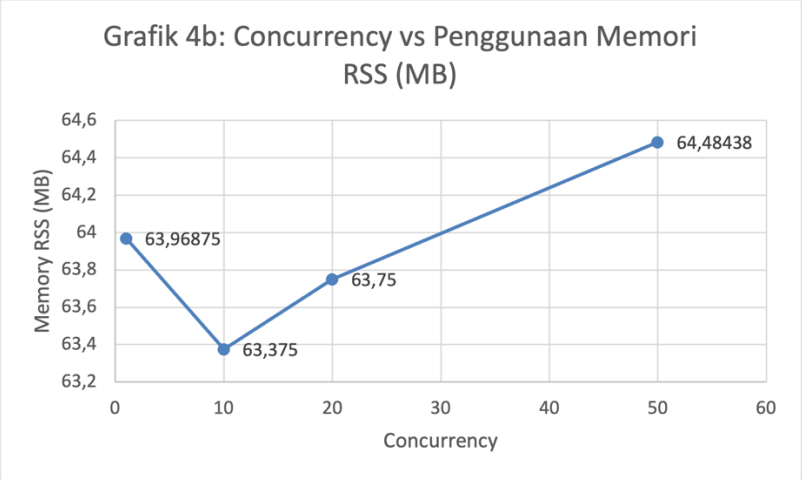
1.4.3 Grafik 3: Concurrency vs Median Failed Requests



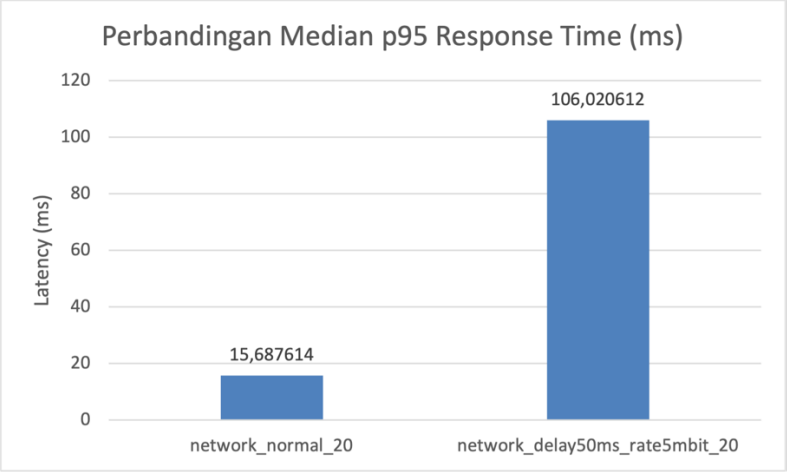
1.4.4 Grafik 4a: Concurrency vs Penggunaan CPU (% 1 Core)

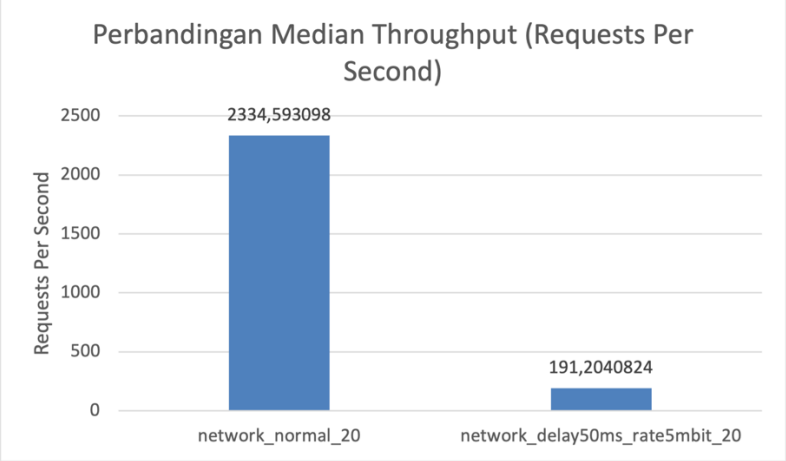


1.4.5 Grafik 4b: Concurrency vs Penggunaan Memori RSS (MB)



1.4.6 Grafik Perbandingan Kondisi Jaringan (N1 vs N2)





1.5 Pembahasan dan Jawaban Pertanyaan Evaluasi ETS

1.5.1 Q1: Perubahan Average, p50, p95, dan p99 Response Time

Seluruh metrik waktu respons (response time) mengalami peningkatan secara bertahap seiring bertambahnya beban concurrency:

- Average Latency: Meningkat dari 0,16 ms (C=1), 0,81 ms (C=10), 1,52 ms (C=20), hingga 3,66 ms (C=50).
- p50 Latency (Median): Meningkat dari 0,13 ms (C=1) hingga 2,65 ms (C=50).
- p95 Latency: Berada pada nilai sangat rendah pada C=1 (0,25 ms), C=10 (1,27 ms), dan C=20 (2,64 ms), kemudian melonjak tajam ke 12,60 ms pada C=50 (kenaikan ~377% dari C=20).
- p99 Latency (Tail Latency): Naik signifikan dari 0,41 ms (C=1) hingga 29,45 ms pada C=50, membuktikan bahwa kelompok request paling lambat mengalami antrean waktu tunggu yang signifikan pada saat concurrency mencapai 50.

1.5.2 Q2: Titik Peningkatan p95 Terjelas dan Perubahan RPS

- Titik Lonjakan p95: Peningkatan p95 terjadi paling drastis pada transisi dari Concurrency 20 ke Concurrency 50 (melonjak dari 2,64 ms ke 12,60 ms).
- Perubahan Throughput (RPS): Pada saat p95 melonjak tajam tersebut, nilai RPS praktis sudah tidak mengalami peningkatan yang berarti. Dari C=20 (13.071,94 RPS) ke C=50 (13.352,66 RPS), RPS hanya naik tipis sebesar +2,1%. Hal ini menandakan sistem telah mencapai batas kapasitas jenuh (saturation point), di mana penambahan concurrency hanya menghasilkan penumpukan antrean (queueing delay) tanpa menambah output pemrosesan.

1.5.3 Q3: Perubahan Failed Requests

Jumlah failed requests bernilai 0 (0,00% error) pada seluruh skenario concurrency (1, 10, 20, dan 50) di setiap pengulangan pengujian. Seluruh 12.000 request (1.000 request × 3 repetisi × 4 skenario) berhasil diproses dan dilayani dengan kode status HTTP 200 OK tanpa adanya koneksi yang terputus atau ditolak.

1.5.4 Q4: Perubahan CPU Server dan Analisis Bottleneck

- Perubahan CPU: Total waktu CPU (CPU time) stabil di kisaran 97–108 ms per run. Namun, persentase CPU relatif satu core meningkat dari 64,18% (C=1) menjadi 125,57% (C=10), 128,90% (C=20), dan 130,26% (C=50). Angka di atas 100% ini terjadi karena proses Node.js memanfaatkan multi-threading internal (thread pool libuv dan proses background V8) pada CPU multi-core mesin penguji.
- Analisis Bottleneck: Data menunjukkan bahwa pemrosesan CPU telah mendekati batas jenuhnya di kisaran ~130%, sehingga throughput mendatar. Namun, data CPU proses Node.js ini belum cukup untuk menyatakan CPU sebagai satu-satunya bottleneck mutlak tanpa bukti metrik sistem operasi tambahan (seperti context switching, CPU steal, atau event loop lag).

1.5.5 Q5: Analisis Penggunaan RAM RSS vs Heap Used

- Memory RSS (Resident Set Size): Total memori fisik RAM yang dialokasikan oleh sistem operasi untuk proses Node.js secara keseluruhan (termasuk biner C++, thread pool, call stack, dan buffer soket jaringan).
- Heap Used: Porsi memori di dalam engine V8 yang secara aktif dialokasikan untuk menyimpan objek, array, dan variabel aplikasi JavaScript.
- Kondisi Stabilitas: Nilai median Memory RSS sangat stabil pada rentang 63,38 MB – 64,48 MB, dan Heap Used stabil di kisaran 6,51 MB – 6,72 MB di seluruh skenario, membuktikan tidak ada kebocoran memori (memory leak).

1.5.6 Q6: Arti Cache Hits, Misses, Hit Ratio dan Efektivitas Performa

- Cache Misses (6): Request awal untuk 6 kategori produk unik (electronics, books, clothing, home, grocery, sports) yang belum tersimpan di memori cache, sehingga harus membaca data sumber sebelum disimpan.
- Cache Hits (994): Sebanyak 994 request berikutnya langsung dilayani dari memori cache ber-TTL 15 detik tanpa perlu mengambil ulang data produk.
- Cache Hit Ratio (99,4%): Rasio efektivitas pemenuhan dari cache, dihitung dari $\frac{994}{994+6} = 0{,}994$
- Efektivitas Performa: Nilai hit ratio yang tinggi membuktikan mekanisme cache bekerja dengan benar, namun belum cukup untuk membuktikan secara empiris bahwa cache meningkatkan performa. Diperlukan pengujian pembanding (baseline benchmark) dengan kondisi cache dimatikan (non-cache) untuk membandingkan selisih latensi dan throughput secara nyata.

1.5.7 Q7: Metrik Filesystem Read/Write vs Disk Throughput (MB/s)

- Hasil Pengukuran: Nilai server_fs_read_ops_delta dan server_fs_write_ops_delta bernilai 0, yang membuktikan seluruh katalog produk dan cache dilayani murni dari memori RAM (in-memory) tanpa operasi I/O disk fisik selama pengujian.
- Alasan Perbedaan: Metrik delta operasi sistem berkas adalah pencacah jumlah panggilan fungsi sistem (system call counter) pada level proses, bukan kecepatan transfer data fisik disk dalam Megabytes per detik (MB/s) dan bukan disk latency.

1.5.8 Q8: Perbandingan Kondisi Jaringan (N1 vs N2)

Scenario	requests_per_second	p95_latency_ms	failed_requests
N1	2334,593098446	15,687614	0
N2	191,20408238383	106,020612	0

- Perubahan p95 Latency: Pada skenario normal (N1), p95 bernilai 15,69 ms, lalu melonjak tajam menjadi 106,02 ms pada skenario pembatasan N2 (naik +575,7%) akibat penundaan 50 ms dan antrean pada bandwidth 5 Mbit/s.
- Perubahan Throughput (RPS): Nilai RPS anjlok drastis dari 2.334,59 RPS (N1) menjadi 191,20 RPS (N2) (turun -91,8%) karena tertahannya pipa transmisi keluar.
- Failed Requests: Tetap bernilai 0 (100% sukses).

1.5.9 Q9: Analisis Kenaikan Response Time Tanpa Kenaikan CPU

Jika latensi p95 naik tetapi pemakaian CPU server tidak meningkat, terdapat dua kemungkinan penyebab:

1. Kendala Transmisi Jaringan (Network Latency / Throttling): Paket respons tertahan dalam buffer TCP antarmuka jaringan atau mengalami antrean transmisi di saluran fisik. Server telah selesai memproses request (CPU idle), tetapi pengiriman respons ke klien tertunda.
   - Data tambahan yang dibutuhkan: Metrik waktu Round-Trip Time (RTT), packet drop rate, TCP retransmission counter, dan utilisasi bandwidth antarmuka jaringan.
2. Antrean Koneksi Masuk (Connection Queue / Event Loop Delay): Request tertahan pada antrean soket sistem operasi (socket listen backlog) sebelum sempat diambil (accept) oleh thread pemroses server.
   - Data tambahan yang dibutuhkan: Metrik kedalaman antrean soket OS (netstat / ss accept queue backlog) dan metrik waktu penundaan event loop (event loop lag).

1.5.10 Q10: Skenario Paling Sesuai dan Analisis Trade-off

1. Skenario Rekomendasi: Concurrency 10 hingga 20.
2. Analisis Trade-Off: Pada Concurrency 20, sistem mencapai throughput mendekati puncak (13.071,94 RPS atau 98% dari throughput maksimal) dengan latensi yang tetap sangat rendah (p95 hanya 2,64 ms), konsumsi CPU stabil, alokasi RAM hemat (~63 MB), dan 0 error. Sebaliknya, Concurrency 50 hanya memberikan kenaikan throughput marjinal sebesar 2,1%, namun harus mengorbankan lonjakan latensi p95 sebesar 377% (menjadi 12,60 ms).

1.5.11 Q11: Keterbatasan Pengujian Laboratorium

1. Pengujian Loopback (Localhost): Pengujian dijalankan pada lingkungan virtual lokal, sehingga meniadakan latensi propagasi jaringan riil, jitter, packet loss, dan variasi kualitas koneksi internet.
2. Karakteristik Data Sintetis In-Memory: Katalog produk disimpan murni di memori RAM tanpa melibatkan database riil (seperti PostgreSQL/MySQL) yang memiliki overhead koneksi TCP dan I/O disk.
3. Pola Akses Beban yang Homogen: Request didistribusikan secara rotasi bergilir rata pada 6 kategori, berbeda dengan kondisi produksi nyata di mana pola akses pengguna bersifat asimetris (skewed traffic / long-tail distribution).

1.6 Kesimpulan

Berdasarkan keseluruhan hasil pengujian eksperimental, API TokoKilat menunjukkan kinerja dan efisiensi skalabilitas optimal pada rentang konkurensi 10 hingga 20 dengan capaian throughput mencapai ~13.000 RPS serta latensi p95 yang terjaga sangat rendah di bawah 2,7 ms tanpa adanya kegagalan request. Peningkatan beban ke konkurensi 50 memicu saturasi kapasitas pemrosesan di mana throughput mengalami stagnasi (hanya bertambah marjinal 2,1%) dan latensi p95 melonjak tajam hingga 12,60 ms akibat penumpukan antrean permintaan (queueing delay). Di samping itu, simulasi gangguan jaringan (N2) membuktikan secara nyata bahwa degradasi saluran transmisi (delay 50 ms dan limitasi bandwidth 5 Mbit/s) secara masif memangkas throughput sebesar 91,8% (turun ke 191,20 RPS) dan mendongkrak latensi menjadi 106,02 ms yang selaras dengan kaidah Hukum Little, menegaskan bahwa performa akhir yang dirasakan pengguna sangat dipengaruhi oleh stabilitas jaringan transmisi di samping kecepatan komputasi in-memory cache server.